

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO3:Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL : 3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:25

Duration: 1 Hour

CASE STUDY

Code of Conduct:

I U.Divya/192372277 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Divya

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Designing a Grammar for Arithmetic Expressions

A compiler development team is creating a parser for a calculator application that supports addition, subtraction, multiplication, division, and parentheses. The initial grammar produces ambiguity for expressions such as $a+b*c$.

Case Study Question:

Develop an optimized context-free grammar (CFG) that correctly enforces operator precedence and associativity for the given arithmetic expressions. Explain how your grammar removes ambiguity and supports efficient parsing.

CASE STUDY 1: Designing a Grammar for Arithmetic Expressions

Problem Understanding

The calculator application must parse arithmetic expressions containing **addition (+), subtraction (-), multiplication (*), division (/), and parentheses**. The initial grammar is ambiguous because an expression such as $a+b*c$ can have more than one parse tree. An optimized CFG is therefore required to enforce **operator precedence and associativity** correctly.

Optimized CFG

The following grammar separates expressions into different levels according to operator precedence:

$$\begin{aligned} E &\rightarrow E + T \mid E - T \mid T \\ T &\rightarrow T * F \mid T / F \mid F \\ F &\rightarrow (E) \mid \text{id} \end{aligned}$$

Where:

- E = Expression
- T = Term
- F = Factor
- id = identifier or number

Operator Precedence

Highest precedence $\rightarrow$ Parentheses

$\downarrow$

Multiplication / Division

$\downarrow$

Lowest precedence $\rightarrow$ Addition / Subtraction

SIMATS ENGINEERING ASSESSMENT TOOLS

Problem with the Initial Ambiguous Grammar

An ambiguous grammar could be:

$$\begin{aligned} E \rightarrow & E + E \\ & | E - E \\ & | E * E \\ & | E / E \\ & | (E) \\ & | id \end{aligned}$$

For:

$$a + b * c$$

the grammar may interpret it as either:

$$(a + b) * c$$

or

$$a + (b * c)$$

This creates ambiguity.

How the Optimized Grammar Removes Ambiguity

The optimized grammar divides the expression into:

$$E \rightarrow \text{addition/subtraction level}$$
$$T \rightarrow \text{multiplication/division level}$$
$$F \rightarrow \text{parentheses/operands level}$$

Therefore:

$$a + b * c$$

is parsed as:

$$a + (b * c)$$

because multiplication has higher precedence than addition.

The left-recursive productions also provide **left associativity**.

For example:

SIMATS ENGINEERING ASSESSMENT TOOLS

$a - b - c$

is interpreted as:

$(a - b) - c$

rather than:

$a - (b - c)$

Example Derivation

For:

$a + b * c$

the derivation is:

E

$\rightarrow E + T$

$\rightarrow T + T$

$\rightarrow F + T$

$\rightarrow id + T$

$\rightarrow id + T * F$

$\rightarrow id + F * F$

$\rightarrow id + id * id$

Therefore:

$a + b * c$

is correctly parsed as:

$a + (b * c)$

Efficiency Improvement

The grammar clearly separates different precedence levels, reducing ambiguity and unnecessary parsing alternatives. This allows a parser to analyze arithmetic expressions more systematically and reduces the number of conflicting parsing decisions.

Conclusion

SIMATS ENGINEERING ASSESSMENT TOOLS

The optimized CFG correctly handles **precedence, associativity, and parentheses**. It eliminates the ambiguity of the original grammar and provides a clear structure for efficient arithmetic expression parsing.

2. Mini Programming Language Parser Development

A startup is designing a small scripting language for automating IoT devices. The language supports variable declarations, conditional statements, and loops. However, the parser frequently reports syntax conflicts during compilation.

Case Study Question:

Construct a CFG for the scripting language constructs below and optimize it to eliminate left recursion and reduce parsing conflicts. Demonstrate how the optimized grammar improves parser efficiency.

CASE STUDY 2: Mini Programming Language Parser Development

Problem Understanding

The IoT scripting language supports **variable declarations, conditional statements, and loops**. The original grammar produces syntax conflicts, especially because of left recursion and multiple possible parsing choices. The grammar should therefore be simplified and transformed into a form suitable for efficient parsing.

Language Constructs

The language supports:

Variable declaration
Conditional statement
Loop statement
Assignment
Expression

Initial Grammar

A simple grammar may contain left recursion such as:

$$\text{StmtList} \rightarrow \text{StmtList Stmt} \mid \text{Stmt}$$

This can cause problems for top-down parsers.

Optimized CFG

The grammar can be rewritten as:

SIMATS ENGINEERING ASSESSMENT TOOLS

Program $\rightarrow$ StmtList

StmtList $\rightarrow$ Stmt StmtList $\mid \epsilon$

Stmt $\rightarrow$ Decl
| Assign
| IfStmt
| WhileStmt

Decl $\rightarrow$ type id ;

Assign $\rightarrow$ id = Expr ;

IfStmt $\rightarrow$ if (Expr) { StmtList }

WhileStmt $\rightarrow$ while (Expr) { StmtList }

Expr $\rightarrow$ id
| number
| id op id

type $\rightarrow$ int $\mid$ float $\mid$ boolean

op $\rightarrow$ + $\mid$ - $\mid$ * $\mid$ / $\mid$ < $\mid$ > $\mid$ ==

Here, ϵ represents the empty string.

Example

Consider:

```
int temperature;  
temperature = 25;
```

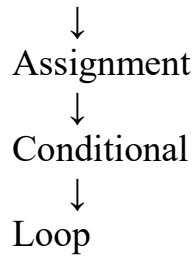
```
if (temperature > 30) {  
    temperature = 30;  
}
```

```
while (temperature < 40) {  
    temperature = temperature + 1;  
}
```

The grammar identifies:

Declaration

SIMATS ENGINEERING ASSESSMENT TOOLS



Removal of Left Recursion

Original:

$\text{StmtList} \rightarrow \text{StmtList Stmt} \mid \text{Stmt}$

Optimized:

$\text{StmtList} \rightarrow \text{Stmt StmtList} \mid \epsilon$

The optimized version does not begin with StmtList on the right-hand side, so it avoids direct left recursion.

How It Reduces Parsing Conflicts

The grammar gives separate productions for:

- declarations
- assignments
- conditions
- loops

Therefore, the parser can identify the type of statement based on its starting token.

For example:

int $\rightarrow$ Declaration
if $\rightarrow$ Conditional
while $\rightarrow$ Loop
identifier $\rightarrow$ Assignment

This reduces unnecessary alternatives and makes predictive parsing easier.

Efficiency Improvement

Removing left recursion prevents infinite recursion in top-down parsers. Separating statement types also reduces parsing conflicts and allows the parser to process IoT scripting statements more efficiently.

SIMATS ENGINEERING ASSESSMENT TOOLS

Conclusion

The optimized CFG provides a clear structure for variable declarations, assignments, conditions, and loops. Removing left recursion and unnecessary alternatives improves parser efficiency and reduces syntax conflicts.

3. Grammar Optimization for SQL Query Validation

A database company is developing a query validation tool for simplified SQL commands such as SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing time.

Case Study Question:

Develop a CFG for simplified SQL query syntax and optimize the grammar by removing unnecessary productions and ambiguity. Discuss how the optimized grammar helps in faster syntax analysis.

CASE STUDY 3: Grammar Optimization for SQL Query Validation

Problem Understanding

The database company needs a syntax validator for simplified SQL queries containing SELECT, FROM, and WHERE. The original grammar contains redundant productions that increase parsing complexity. The grammar should be simplified while maintaining correct SQL query structure.

Optimized CFG

Query $\rightarrow$ SELECT ColumnList FROM Table WhereClause

ColumnList $\rightarrow$ id ColumnTail

ColumnTail $\rightarrow$, id ColumnTail | ϵ

WhereClause $\rightarrow$ WHERE Condition | ϵ

Condition $\rightarrow$ id Operator Value

Operator $\rightarrow$ = | < | > | <= | >= | <>

Value $\rightarrow$ id | number | string

Table $\rightarrow$ id

Example Valid Query

SIMATS ENGINEERING ASSESSMENT TOOLS

SELECT name, age FROM students WHERE age > 18

The grammar processes it as:

Query
↓
SELECT
↓
ColumnList
↓
FROM
↓
Table
↓
WHERE
↓
Condition

Example Derivation

Query
→ SELECT ColumnList FROM Table WhereClause

→ SELECT id ColumnTail FROM id WhereClause

→ SELECT name , id FROM students WhereClause

→ SELECT name , age FROM students WHERE Condition

→ SELECT name , age FROM students WHERE age > 18

Handling Optional WHERE

The production:

WhereClause → WHERE Condition | ϵ

allows both:

SELECT name FROM students

and:

SELECT name FROM students WHERE age > 18

Therefore, WHERE is optional.

SIMATS ENGINEERING ASSESSMENT TOOLS

Removal of Redundant Productions

Instead of creating separate productions for every possible query, reusable components are defined:

ColumnList
ColumnTail
WhereClause
Condition
Operator
Value

This avoids unnecessary repetition.

How the Optimized Grammar Improves Parsing

The grammar provides a fixed query structure:

SELECT → Columns → FROM → Table → Optional WHERE

This allows the parser to quickly identify the expected next component.

For example:

SELECT
↓
ColumnList
↓
FROM
↓
Table
↓
WHERE (optional)

This reduces unnecessary parsing alternatives and improves syntax validation efficiency.

Invalid Query Example

SELECT FROM students

is rejected because ColumnList requires at least one column:

ColumnList → id ColumnTail

Similarly:

SIMATS ENGINEERING ASSESSMENT TOOLS

SELECT name students

is rejected because FROM is required after the column list.

Conclusion

The optimized SQL CFG removes redundant productions and clearly defines the order of SQL components. It improves syntax validation by reducing unnecessary parsing choices and making the grammar easier to process.

4. Natural Language Processing Chatbot

A chatbot system processes simple English sentences such as:

Subject + Verb + Object

Example: “Students learn TOC.”

The initial CFG incorrectly accepts grammatically invalid sentences.

Case Study Question:

Design a CFG for the chatbot sentence structure and optimize it to improve language validation accuracy. Explain how the grammar ensures correct sentence formation and rejects invalid inputs.

CASE STUDY 4: Natural Language Processing Chatbot

Problem Understanding

The chatbot must validate simple English sentences using the structure **Subject + Verb + Object**. The original grammar accepts grammatically incorrect combinations. A more restricted CFG is required to ensure that the subject, verb, and object are correctly formed.

Basic Sentence Structure

The required structure is:

Sentence → Subject Verb Object

Optimized CFG

Sentence → Subject Verb Object

Subject → Noun
 | Determiner Noun

Object → Noun
 | Determiner Noun

SIMATS ENGINEERING ASSESSMENT TOOLS

Verb → TransitiveVerb

Noun → Students

- | Teachers
- | Students
- | Books
- | TOC

Determiner → the

- | a

TransitiveVerb → learn

- | study
- | read
- | teach

Example Valid Sentence

Students learn TOC.

Derivation:

Sentence

→ Subject Verb Object

→ Noun Verb Noun

→ Students learn TOC

Therefore:

Students learn TOC.

is accepted.

Another Valid Sentence

The students study TOC.

Derivation:

Sentence

→ Subject Verb Object

→ Determiner Noun Verb Noun

→ The students study TOC

SIMATS ENGINEERING ASSESSMENT TOOLS

Invalid Sentence

Consider:

Students TOC learn.

The required structure is:

Subject + Verb + Object

but the sentence has:

Subject + Object + Verb

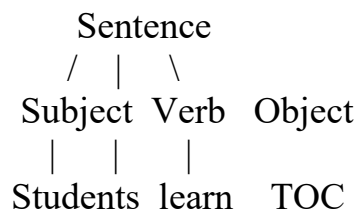
Therefore, it is rejected.

Another invalid sentence:

Learn students TOC.

is rejected because the sentence does not begin with a valid subject.

Parse Structure



This structure clearly shows:

Subject → Students

Verb → learn

Object → TOC

How the Grammar Improves Accuracy

The grammar restricts the sentence structure to:

Subject → Verb → Object

Only words belonging to the appropriate grammar categories can appear in each position. Therefore, incorrectly ordered sentences can be rejected during syntax validation.

Conclusion

SIMATS ENGINEERING ASSESSMENT TOOLS

The optimized CFG provides a simple structure for validating English sentences. It ensures that the chatbot accepts correctly ordered sentences such as "**Students learn TOC**" and rejects sentences with incorrect grammatical structure.

5. Parser Design for HTML-like Markup Language

A web editor company is building a parser for a simplified markup language with opening and closing tags. The existing grammar fails to validate nested tags correctly.

Case Study Question:

Develop and optimize a CFG for validating properly nested tags in the markup language. Show how the grammar supports recursive nesting and improves parsing reliability.

CASE STUDY 5: Parser Design for HTML-like Markup Language

Problem Understanding

The web editor needs to validate opening and closing tags in a simplified markup language. The main problem is ensuring that nested tags are properly matched. A recursive CFG can represent nested structures and ensure that every opening tag has a corresponding closing tag.

Optimized CFG

For a simplified markup language using fixed tags:

Document $\rightarrow$ Element

Element $\rightarrow$ $\langle\text{html}\rangle$ Element $\langle\text{/html}\rangle$
| $\langle\text{body}\rangle$ Element $\langle\text{/body}\rangle$
| $\langle\text{p}\rangle$ Text $\langle\text{/p}\rangle$
| Text

Text $\rightarrow$ word Text | word

For a more general representation:

Document $\rightarrow$ ElementList

ElementList $\rightarrow$ Element ElementList | ϵ

Element $\rightarrow$ $\langle\text{tag}\rangle$ ElementList $\langle\text{/tag}\rangle$
| Text

Text $\rightarrow$ word Text | word

SIMATS ENGINEERING ASSESSMENT TOOLS

Here:

`<tag>`

represents an opening tag and:

`</tag>`

represents the corresponding closing tag.

Example Valid Markup

```
<html>
  <body>
    <p>
      Hello
    </p>
  </body></html>
```

The recursive structure is:

```
<html>
|
+-- <body>
    |
    +-- <p>
        |
        +-- Text
```

The closing tags are matched in reverse order:

```
<p>    → </p>
<body> → </body>
<html> → </html>
```

Why Recursion Is Required

Consider:

```
<html>
  <body>
    <p>Hello</p>
  </body></html>
```

The `<html>` element contains a `<body>` element, which contains a `<p>` element.

The grammar recursively represents this:

SIMATS ENGINEERING ASSESSMENT TOOLS

Element

→ <html> ElementList </html>

ElementList

→ Element

Element

→ <body> ElementList </body>

ElementList

→ Element

Element

→ <p> Text </p>

Thus, nested elements can be represented naturally.

Invalid Markup Example

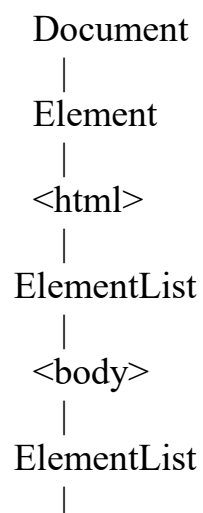
```
<html>
  <body>
    <p>Hello</body>
  </p></html>
```

This is rejected because the tags are not properly nested.

Correct nesting requires:

```
<p> ... </p>
<body> ... </body>
<html> ... </html>
```

Parse Structure



SIMATS ENGINEERING ASSESSMENT TOOLS

<p>
|
Text

How the Grammar Improves Parsing Reliability

The recursive production:

Element $\rightarrow$ <tag> ElementList </tag>

ensures that an opening tag contains its corresponding closing tag.

It also naturally supports:

Nested elements

↓

Element inside Element

↓

Element inside Element

↓

...

Therefore, the parser can validate deeply nested markup structures.

Conclusion

The recursive CFG correctly represents nested markup elements and ensures that opening and closing tags are properly matched. This improves the reliability of the markup parser and helps detect incorrectly nested tags.

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0– 1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well- organized answer	Understandabl e with minor issues	Somewhat unclear or poorly structured	Difficult to understand